Project work part 3 - Data quality

- Author: Mehdi Jamali

Links

- GitHub repository
- Streamlit app
- Video: The screencast will be recorded and added on 11 November, after the Open-Meteo request limit resets. It could not be completed before today's submission deadline.

Declaration of AI usage

I used the following AI tools while working on this project:

- **GitHub Copilot:**
  - Used for code completion, suggestions, and feedback during development.
  - Also used to help implement and test the Part 3 ERA5 downloader, data-quality analyses, Streamlit pages, and caching safeguards.
- **Gemini:**
  - Used for coding assistance and for finding relevant functions and methods.
- **NotebookLM (Gemini):**
  - Used as a research assistant to answer questions and explain topics related to the project.

I reviewed and adapted the suggestions, and tested the code before using it in the project.

The AI tools were used within the guidelines of "K3 - Full use of AI" at NMBU.

Work log

I worked on Part 3 in the temporary `part3` branch so the changes could remain separate from `main` during peer review. I started by loading the NVE region shapefile with GeoPandas. The file contains the regions `RM`, `RN`, `RS`, `RV`, and `RØ` in a projected coordinate system. I generated reproducible random samples inside each polygon and converted them to longitude and latitude for the weather API. I used 1,000 sample points per region and verified the sample counts and polygon membership. I then mapped the electricity areas as NO1=RS+RØ, NO2=RS, NO3=RM, NO4=RM+RN, and NO5=RV.

The main technical challenge was designing a reliable ERA5 downloader. The first implementation sent repeated list-valued parameters for dates, variables, model, and timezone. Open-Meteo rejected that request format. I corrected it so longitude and latitude are combined arrays with up to 100 coordinates per request, while the other parameters are shared scalar values. Several grid points can represent the same 0.25-degree ERA5 cell, so I request each cell once and weight its contribution by the number of sample points. Successful responses are cached locally, and the implementation can also use the shared MongoDB cache. A five-location full-year test returned 8,784 hourly observations, and mocked tests confirmed the batching, weighting, completeness checks, and cache reuse.

The complete NO3 download was blocked by external API limits. At the final check, 155 of 286 grid cells had full-year responses. The first large requests also produced `ReadTimeout` errors. After the request format and batching were corrected, Open-Meteo reported that the daily request quota had been reached, with completed batches retained in the cache. I added handling for timeouts, HTTP errors, and budget reservations so the application does not present an unhandled traceback or count rejected requests as successful usage. Because the complete series was not available, I did not present partial precipitation or temperature results as final analyses.

For the data-quality work, I implemented the DCT high-pass SATV/SPC precipitation analysis, Local Outlier Factor temperature anomaly detection, STL decomposition, and a spectrogram for reservoir filling. The reservoir STL and spectrogram cell ran successfully. The weather-analysis cell could not run because `weather_no3_2024` was never created after the failed download. The Streamlit app was updated with a reservoir-seasonality page and a weather-outliers page, using tabs, area selection, caching, download controls, and readable API error messages. I tested the analysis functions with mocked data and tested the request and timeout flows locally. I used GitHub Copilot, Gemini, and NotebookLM for explanations and coding suggestions, and reviewed and tested the resulting changes myself. I will submit today without the screencast because the API limit prevented a complete live demonstration; the screencast is planned for 11 November after the limit resets.

Load and check the NVE region shapefile

I keep the five region codes separate so each sampled location can be traced back to its source polygon. A fixed random seed makes the same sample reproducible.

Inspect region names and sample locations

The helper reads the shapefile, checks the region codes, and creates the sample locations.

In [1]:
```
# Find the project root so imports work from either notebook location.
%matplotlib inline
from pathlib import Path
import sys
import pandas as pd

project_root = Path.cwd()
if not (project_root / "part3_data.py").is_file():
    project_root = project_root.parent
if not (project_root / "part3_data.py").is_file():
    raise FileNotFoundError("Could not find part3_data.py in the project.")
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))

# Load the regions and take a reproducible sample from each one.
from part3_data import load_nve_regions, sample_region_locations

regions = load_nve_regions()
region_coordinates = sample_region_locations(
    regions,
    samples_per_region=1000,
    random_seed=42,
)

sample_counts = {code: len(points) for code, points in region_coordinates.items()}
assert set(sample_counts) == {"RM", "RN", "RS", "RV", "RØ"}
assert all(count == 1000 for count in sample_counts.values())
```

```
print(regions[["REGIONNAVN"]])
print("Sampled locations per region:", sample_counts)
```

```
              REGIONNAVN
REGION
RM      Region Midt-Norge
RN            Region Nord
RS             Region Sør
RV            Region Vest
RØ             Region Øst
Sampled locations per region: {'RM': 1000, 'RN': 1000, 'RS': 1000, 'RV': 1000, 'RØ': 1000}
```

Download hourly ERA5 data for NO3

I map the electricity areas to the NVE regions as follows: NO1 uses RS and RØ, NO2 uses RS, NO3 uses RM, NO4 uses RM and RN, and NO5 uses RV. In other words, East is RS+RØ and North is RM+RN. For each region, the downloader starts with 1,000 sampled locations. If several samples fall in the same 0.25-degree ERA5 grid cell, it requests that cell once and weights its contribution by the number of sample points there. Completed API batches and validated full area/year results are cached locally so later runs can reuse them.

> **Weather-data status:** Open-Meteo's request limit prevented the full NO3 download from finishing. The cached responses are incomplete, so the weather-analysis cells below do not contain final results. I have left them this way rather than presenting partial data as complete.

In [3]:
```
from part3_data import download_era5_data

# Request all hourly weather data for 2024; the downloader checks completeness.
weather_no3_2024 = download_era5_data(
    region_identifier="NO3",
    start_date="2024-01-01",
    end_date="2024-12-31",
    samples_per_region=1000,
    random_seed=42,
)
assert len(weather_no3_2024) == 8784
assert weather_no3_2024["time"].is_monotonic_increasing
assert weather_no3_2024["time"].is_unique
assert weather_no3_2024[["precipitation", "temperature_2m"]].notna().all().all()

# Save the complete series locally so future runs can reuse it.
weather_cache_file = project_root / "data" / "era5_cache" / "NO3_ERA5_2024.csv"
weather_cache_file.parent.mkdir(parents=True, exist_ok=True)
weather_no3_2024.to_csv(weather_cache_file, index=False)

print(f"Downloaded {len(weather_no3_2024):,} hourly observations.")
print(f"Cached complete area series at {weather_cache_file}.")
weather_no3_2024.head()
```

```
---------------------------------------------------------------------------
Era5ApiBudgetExceeded                     Traceback (most recent call last)
Cell In[3], line 4
      1 from part3_data import download_era5_data
      2
      3 # Request all hourly weather data for 2024; the downloader checks completeness.
----> 4 weather_no3_2024 = download_era5_data(
      5     region_identifier="NO3",
      6     start_date="2024-01-01",
      7     end_date="2024-12-31",

File ~\Desktop\IND320\IND320-Mehdij1511\part3_data.py:682, in download_era5_data(region_identi
fier, start_date, end_date, samples_per_region, random_seed)
    674 grid_weights = Counter(
    675     (
    676         round(longitude / ERA5_GRID_DEGREES) * ERA5_GRID_DEGREES,
   (...)    679     for longitude, latitude in coordinates
    680 )
    681 grid_coordinates = list(grid_weights)
--> 682 data = _download_era5_locations(
    683     grid_coordinates,
    684     start.strftime("%Y-%m-%d"),
    685     end.strftime("%Y-%m-%d"),
    686     location_weights=list(grid_weights.values()),
    687 )
    688 if (
    689     not pd.DatetimeIndex(data["time"]).equals(expected_times)
    690     or not data[["precipitation", "temperature_2m"]].notna().all().all()
    691 ):
    692     raise ValueError(
    693         "The ERA5 area series is incomplete; it will not be cached as complete."
    694     )

File ~\Desktop\IND320\IND320-Mehdij1511\part3_data.py:494, in _download_era5_locations(coordin
ates, start_date, end_date, batch_size, location_weights)
    492 if not cached:
    493     last_request_started = monotonic()
--> 494 locations = _request_era5_batch(payload, cache_dir)
    495 if len(locations) != location_count:
    496     raise ValueError(
    497         "Open-Meteo returned a different number of locations "
    498         f"({len(locations)}) than requested ({location_count})."
    499     )

File ~\Desktop\IND320\IND320-Mehdij1511\part3_data.py:275, in _request_era5_batch(payload, cac
he_dir)
    273 if "daily api request limit" in response_detail.lower():
    274     _release_era5_call_budget(reservation)
--> 275     raise Era5ApiBudgetExceeded(
    276         "Open-Meteo reported that its daily API limit has been reached. "
    277         f"Completed batches are cached.{retry_message} Retry manually "
    278         "after the limit resets."
    279     )
    280 _release_era5_call_budget(reservation)
    281 detail_message = (
    282     f" Server response: {response_detail}" if response_detail else ""
    283 )

Era5ApiBudgetExceeded: Open-Meteo reported that its daily API limit has been reached. Complete
d batches are cached. Retry-After: 10921. Retry manually after the limit resets.
```

Precipitation outliers and temperature anomalies

The precipitation analysis uses a high-pass DCT to calculate SATV and robust SPC limits, while the temperature analysis identifies anomalies with Local Outlier Factor. SATV is used to classify outliers but is not plotted.

In [4]:
```
from IPython.display import display
from part3_analysis import plot_precipitation_outliers, plot_temperature_anomalies

# Use DCT-derived SATV to classify outliers; the plot shows precipitation and limits.
precipitation_figure, precipitation_summary, precipitation_outliers = (
    plot_precipitation_outliers(
        weather_no3_2024,
        frequency_cutoff=52,
        n_standard_deviations=3,
    )
)
assert precipitation_summary["observations"] == len(weather_no3_2024)
assert len(precipitation_outliers) == precipitation_summary["outliers"]
display(precipitation_figure)
display(pd.Series(precipitation_summary, name="Precipitation SPC summary"))
display(precipitation_outliers.head(10))

# Use Local Outlier Factor to flag unusual temperatures (1% expected by default).
temperature_figure, temperature_summary, temperature_anomalies = (
    plot_temperature_anomalies(weather_no3_2024, contamination=0.01)
)
assert temperature_summary["observations"] == len(weather_no3_2024)
assert len(temperature_anomalies) == temperature_summary["anomalies"]
display(temperature_figure)
display(pd.Series(temperature_summary, name="Temperature LOF summary"))
display(temperature_anomalies.head(10))
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[4], line 7
      3
      4 # Use DCT-derived SATV to classify outliers; the plot shows precipitation and limits.
      5 precipitation_figure, precipitation_summary, precipitation_outliers = (
      6     plot_precipitation_outliers(
----> 7         weather_no3_2024,
      8         frequency_cutoff=52,
      9         n_standard_deviations=3,
     10     )

NameError: name 'weather_no3_2024' is not defined
```

Execution note: This analysis cell could not run because the preceding ERA5 download failed. See the Work log for the API timeout and quota details.

Reservoir-filling STL and spectrogram

Weekly filling levels for electricity area 3 (NO3) are retrieved from Magasinstatistikk. STL and the spectrogram use the same time series.

In [5]:
```
import matplotlib.pyplot as plt
from nve_data import load_reservoir_data
from part3_analysis import plot_spectrogram, plot_stl_decomposition

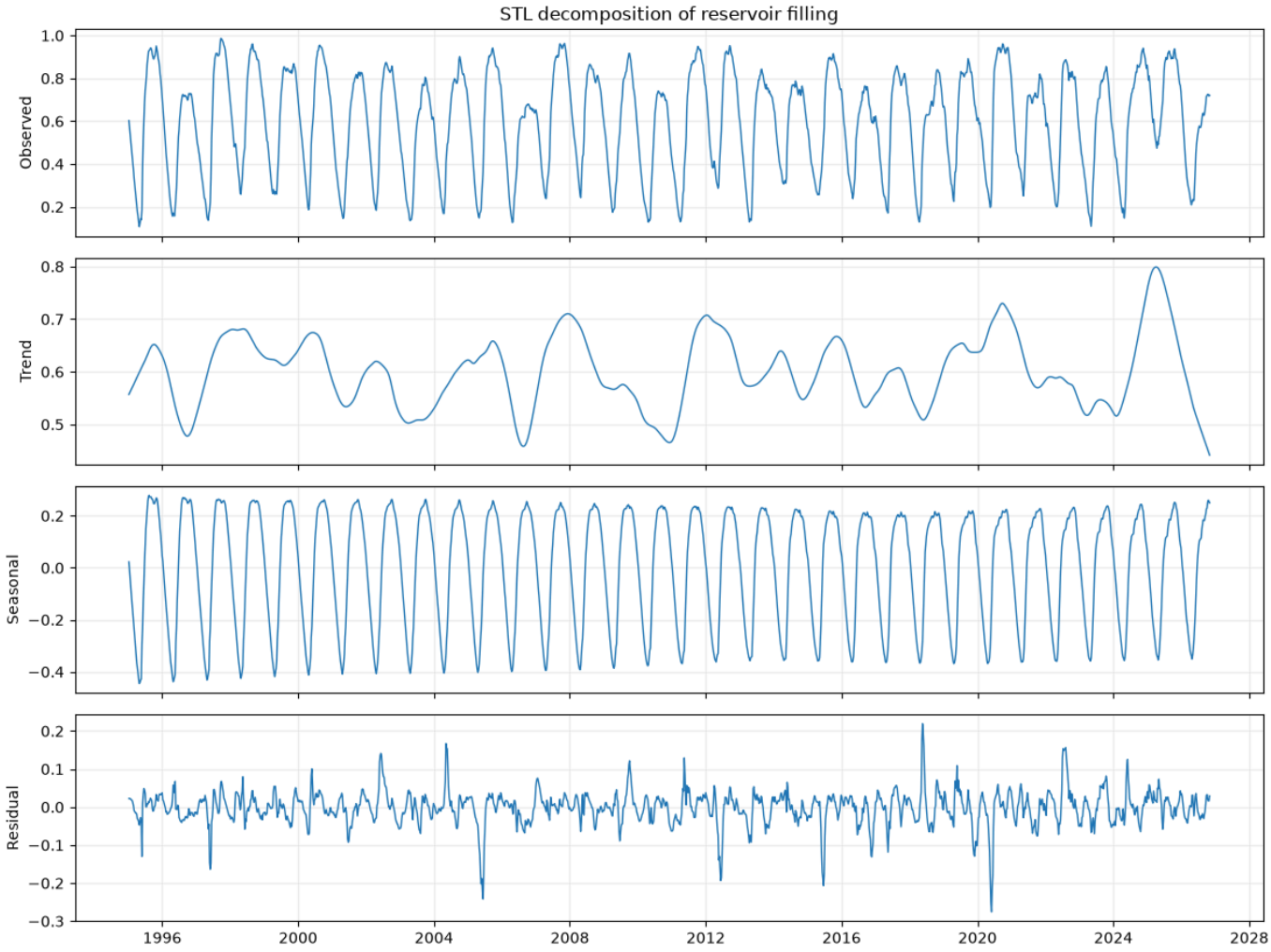
# Use the NO3 reservoir-filling series (area 3) for this analysis.
reservoir_data = load_reservoir_data()
no3_reservoir_data = reservoir_data.loc[
    (reservoir_data["omrType"] == "EL") & (reservoir_data["omrnr"] == 3)
].sort_values("dato_Id")
```

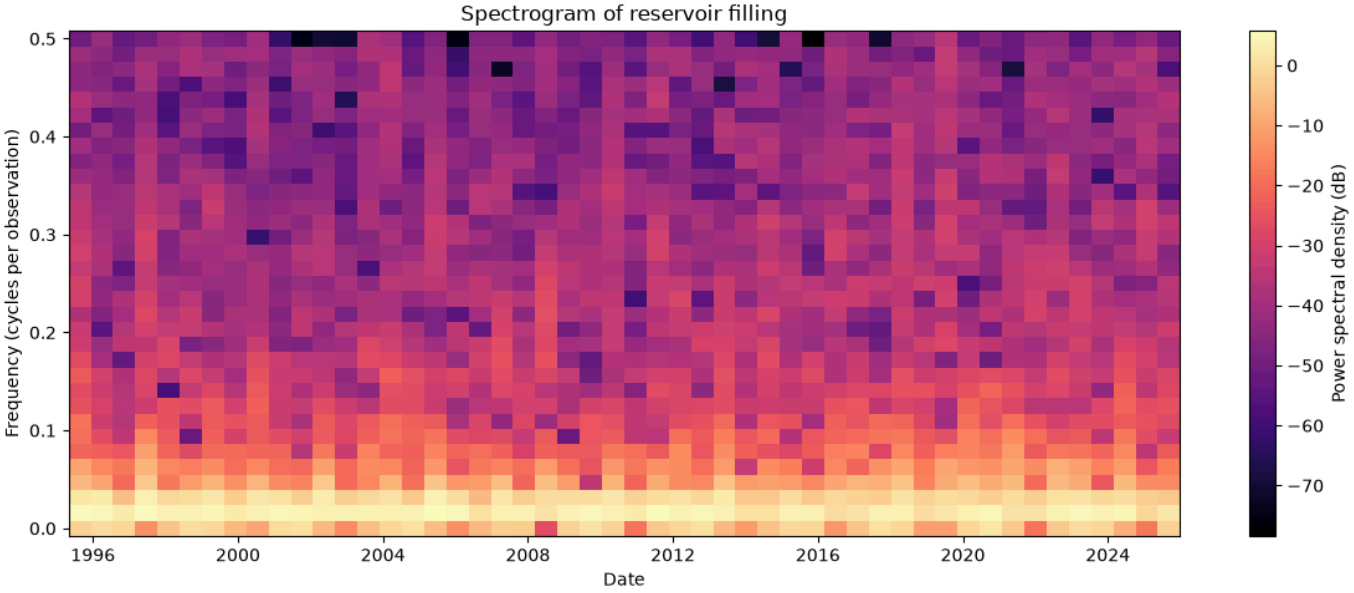
```python
no3_filling = no3_reservoir_data.set_index("dato_Id")["fyllingsgrad"]

stl_figure, stl_result = plot_stl_decomposition(
    no3_filling,
    seasonal=13,
    trend=53,
    robust=True,
)
assert len(stl_result.trend) == len(no3_filling)
display(stl_figure)
plt.close(stl_figure)

spectrogram_figure = plot_spectrogram(
    no3_filling,
    window_length=64,
    window_overlap=32,
)
assert spectrogram_figure.axes
display(spectrogram_figure)
plt.close(spectrogram_figure)
```

```
2026-11-10 21:59:18.563 No runtime found, using MemoryCacheStorageManager
2026-11-10 21:59:18.565 No runtime found, using MemoryCacheStorageManager
2026-11-10 21:59:18.567 Thread 'MainThread': missing ScriptRunContext! This warning can be ign
ored when running in bare mode.
```




Spectrogram of reservoir filling
Frequency (cycles per observation)
0.5
0.4
0.3
0.2
0.1
0.0
1996
2000
2004
2008
2012
2016
2020
2024
Date
0
−10
−20
−30
−40
−50
−60
−70
Power spectral density (dB)